LINUX COMMANDS (Chapters 1–4)

Name:	Ahmed Suleiman Abuukar	ID:	C6240343

Chapter 1: Introduction to Linux and Ubuntu

1.1 uname

Definition

Displays general system and Linux kernel information running on the system, including kernel release version, hardware architecture, and machine hostname.

Why students use this command

Use it to verify the active Linux kernel version and confirm system architecture before installing packages or troubleshooting hardware compatibility.

Syntax

```
uname [OPTION]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-r`	Kernel release	Function: Displays the version number of the Linux kernel running on the system. Example: **`uname -r`**
`-m`	Machine hardware name	Function: Displays the system hardware architecture (e.g., x86_64). Example: **`uname -m`**
`-n`	Network node hostname	Function: Prints the network hostname of the machine. Example: **`uname -n`**
`-a`	All system information	Function: Prints all available system and kernel details in one line. Example: **`uname -a`**

Practice terminal — test commands and actual output

Ubuntu terminal | uname | lab practice

```
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ uname -r
 7.0.0-38-generic
```

1.2 cat /etc/os-release

Definition

Displays information and details about the Ubuntu operating system release stored in the **`/etc/os-release`** configuration file.

Why students use this command

Use it to check the exact Ubuntu distribution name, version ID, and codename on a local or remote Linux machine.

Syntax

```
cat /etc/os-release
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`/etc/os-release`	OS identification file	Function: Reads key-value pairs describing the operating system version. Example: **`cat /etc/os-release`**

Practice terminal — test commands and actual output

Ubuntu terminal | cat /etc/os-release | lab practice

```
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04.1 LTS" NAME="Ubuntu" VERSION_ID="26.04" VERSION="26.04.1
LTS (Resolute Raccoon)" VERSION_CODENAME=resolute ID=ubuntu ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/" SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute LOGO=ubuntu-logo
```

1.3 whoami

Definition

Outputs the username of the currently logged-in user associated with the active terminal session.

Why students use this command

Use it to confirm which user account is currently executing commands, especially after switching accounts or using administrative shells.

Syntax

```
whoami
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`--help`	Display usage help	Function: Shows brief usage instructions for the whoami command. Example: **`whoami --help`**

Practice terminal — test commands and actual output

```
ahmed@ahmed-Vmware-Virtual-Platform:~/Desktop$ whoami
ahmed
```

1.4 pwd

Definition

Shows the full absolute path of the directory you are currently working in (Print Working Directory).

Why students use this command

Use it to verify your exact location in the Linux directory hierarchy before creating, moving, or deleting files.

Syntax

```
pwd [OPTION]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-L`	Logical path	Function: Prints the current directory path including symbolic links (default). Example: **`pwd -L`**
`-P`	Physical path	Function: Resolves all symbolic links and prints the physical directory path. Example: **`pwd -P`**

Practice terminal — test commands and actual output

```
• • •  Ubuntu terminal | pwd | lab practice

ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ pwd
/home/ahmed/Desktop
```

Chapter 2: Linux System Architecture, Command Line, and File Management

2.1 cd

Definition

Allows you to change from one directory to another within the Linux filesystem.

Why students use this command

Use it to navigate between your home folder, Desktop, system configuration directories, and project folders.

Syntax

```
cd [DIRECTORY]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
cd	Home directory	Function: Returns directly to the logged-in user's home directory. Example: **cd**
cd ..	Parent directory	Function: Moves one level up to the parent directory. Example: **cd ..**

Practice terminal — test commands and actual output

Ubuntu terminal | cd | lab practice

```
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ cd
ahmed@ahmed-VMware-Virtual-Platform:~$ cd Desktop
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$
```

2.2 ls

Definition

Lists the files and directories inside your current directory or a specified target path.

Why students use this command

Use it to inspect directory contents, check file permissions, verify file sizes, and view inode numbers.

Syntax

```
ls [OPTION]... [FILE]...
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-l`	Long listing format	Function: Displays detailed file information (permissions, owner, size, timestamp). Example: **`ls -l`**
`-lh`	Human-readable sizes	Function: Displays file sizes in a human-readable format (e.g., KB, MB). Example: **`ls -lh`**
`-li`	Inode number listing	Function: Displays the file's Inode number along with link counts. Example: **`ls -li`**

Practice terminal — test commands and actual output

Ubuntu terminal | ls | lab practice

```
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ ls
cns.txt err.txt hard-link.txt num.txt student.tar cns.txt.gz exam.txt image.jpg
numbers.txt student.tcns241.txt file.tar link-shortcut.txt out.txt test1.txt cns245.
txt file1.txt name.txt cns256 file2.txt names.txt renamed-link
 ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ ls -li
total 88 396242 -rw-rw-r-- 1 ahmed ahmed 0 Nov 11 11:59 cns.txt 396246 -rw-rw-r-- 1
ahmed ahmed 28 Nov 11 11:59 cns.txt.gz
529563 drwxrwxr-x 2 ahmed ahmed 4096 Nov 11 11:22 cns256 396217 -rw-rw-r-- 2 ahmed
ahmed 34 Nov 11 12:18 hard-link.txt 396256 lrwxrwxrwx 1 ahmed ahmed 8 Nov 11 12:25
link-shortcut.txt -> test.txt
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ ls -lh
total 88K -rw-rw-r-- 1 ahmed ahmed 0 Nov 11 11:59 cns.txt drwxrwxr-x 2 ahmed ahmed
4.0K Nov 11 11:22 cns256 -rw-rw-r-- 1 ahmed ahmed 10K Nov 11 11:55 file.tar
```

2.3 history, man, --help, and info

Definition

`history` displays a list of the commands you have previously executed. **`man`** displays the manual page or instruction guide for a specified command. **`--help`** is an option used with commands to display brief usage instructions, while **`info`** opens more detailed documentation or information about a command or program.

Why students use this command

Use these commands to recall previous terminal work and read built-in documentation without needing external internet access.

Syntax

```
history | man [COMMAND] | [COMMAND] --help | info [COMMAND]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`history`	Command history list	Function: Displays numbered list of previously executed commands. Example: **`history`**
`man [cmd]`	Manual page	Function: Opens the official instruction guide for a command. Example: **`man ls`**
`--help`	Quick usage summary	Function: Displays brief command syntax and flag descriptions. Example: **`ls --help`**
`info [cmd]`	Detailed documentation	Function: Opens hyperlinked GNU info documentation. Example: **`info bash`**

Practice terminal — test commands and actual output

Ubuntu terminal | history & help | lab practice

```
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ history
32 ls 33 rm file1.txt file2.txt fil1.txt 39 rm test.txt 45 rm -rf fil.2txt 56
clear
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ help
GNU bash, version 5.3.9(1)-release (x86_64-pc-linux-gnu) These shell commands
are defined internally. Type `help' to see this list.
```

2.4 touch, mkdir, rmdir, cp, mv, and rm

Definition

`touch` creates a new empty file or updates a file's timestamp. **`mkdir`** creates a new directory. **`rmdir`** deletes a directory if it is empty. **`cp`** copies files or directories, **`mv`** moves or renames files and directories, and **`rm`** deletes files or directories.

Why students use this command

Use these core file-management commands to organize folders, duplicate data, rename files, and clean up obsolete directories.

Syntax

```
touch [FILE] | mkdir [OPTION] [DIR] | cp [OPTION] [SRC] [DEST] | mv [SRC]
[DEST] | rm [OPTION] [FILE]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`mkdir -p`	Create parent directories	Function: Creates parent directories as needed if they do not already exist. Example: **`mkdir -p dir1/dir2`**
`cp -r`	Recursive directory copy	Function: Copies directories and all their contents recursively. Example: **`cp -r databse files`**
`rm -r`	Recursive deletion	Function: Deletes directories and their contents recursively. Example: **`rm -r files`**
`rmdir`	Remove empty directory	Function: Deletes a directory only if it contains no files. Example: **`rmdir word`**

Practice terminal — test commands and actual output

Ubuntu terminal | touch, mkdir, cp, mv, rm | lab practice

```
      ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ touch file1.txt
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ touch file2.txt
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ mkdir files
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ mkdir databse
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ cp -r databse files
 ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ mv databse files
 ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ rm -r files
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ rmdir word
 ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ rm file1.txt file2.txt
```

Chapter 3: Text Processing, Searching, Archiving, and Editors

3.1 cat, less, head, and tail

Definition

`cat` reads and displays file contents on the screen. **`less`** allows you to view large files page by page, scrolling up and down. **`head`** displays the beginning (first lines) of a file by default, while **`tail`** displays the end (last lines) of a file by default.

Why students use this command

Use these commands to inspect text files, preview headers or footers of datasets, and monitor real-time log updates.

Syntax

```
cat [OPTION] [FILE] | less [FILE] | head -n [NUM] [FILE] | tail -n [NUM] [FILE]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`cat -n`	Number all lines	Function: Numbers all output lines including blank lines. Example: **`cat -n names.txt`**
`cat -s`	Squeeze blank lines	Function: Suppresses repeated empty output lines (input space). Example: **`cat -s names.txt`**
`cat -b`	Number non-blank lines	Function: Numbers only non-empty lines (space with numbers). Example: **`cat -b names.txt`**
`head -n [number]`	Top N lines	Function: Displays a specific number of lines from the top. Example: **`head -n 4 names.txt`**
`tail -n [number]`	Bottom N lines	Function: Displays a specific number of lines from the bottom. Example: **`tail -n 4 names.txt`**
`tail -f`	Follow file growth	Function: Continuously monitors and outputs appended data to a file in real-time. Example: **`tail -f /var/log/syslog`**

Practice terminal — test commands and actual output

```
Ubuntu terminal | cat, head, tail | lab practice
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ cat -n names.txt
1 Cabdullaahi Yuusuf
2 Maxamed Nuur
3 Bashiir Maxamuud
4 Faarax Xasan
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ head -n 4 names.txt
Cabdullaahi Yuusuf Maxamed Nuur Bashiir Maxamuud Faarax Xasan
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ tail -n 4 names.txt
Xasan Jaamac Cali Maxamed Mustaf Maxamed Ismaaciil Xasan
```

3.2 which, find, locate, updatedb, and grep

Definition

`which` searches directories in the PATH environment variable to locate executable programs. **`find`** searches for files and directories dynamically based on various conditions. **`locate`** finds files quickly by searching a prebuilt filename database refreshed by **`updatedb`**. **`grep`** searches for specific text patterns or regular expressions inside files.

Why students use this command

Use these search utilities to locate binaries, filter files by name or byte size, and extract matching lines inside large text files.

Syntax

```
which [CMD] | find [PATH] [EXPRESSION] | locate [PATTERN] | grep [OPTION] [PATTERN] [FILE]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`find . -name '[name]'`	Search by filename	Function: Searches for files by name case-sensitively. Example: **`find -name "*.txt"`**
`find . -type f`	Regular files only	Function: Selects regular files only. Example: **`find . -type f`**
`find . -size +10c`	Filter by file size	Function: Searches for files larger than 10 bytes. Example: **`find -name "*.txt" -size +40c`**
`grep -n`	Show line numbers	Function: Displays matching line numbers alongside matched text. Example: **`grep -n "Axmed" names.txt`**
`grep -i`	Case-insensitive match	Function: Performs case-insensitive pattern matching. Example: **`grep -i "axmed" names.txt`**

Practice terminal — test commands and actual output

Ubuntu terminal | which, find, locate, grep | lab practice

```
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ which ls
/usr/bin/ls
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ find -name "*.txt"
./numbers.txt ./names.txt
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ sudo updatedb
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ locate cns245.txt
/home/ahmed/Desktop/cns245.txt
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ grep "Axmed Cali" names.txt
Axmed Cali
```

3.3 I/O Redirection (>, >>, 2>, <) and Pipe (|)

Definition

Redirection operators control where command input comes from and where standard output or error messages are saved. The pipe operator (**|**) connects the standard output of one command directly to the standard input of another.

Why students use this command

Use redirection and pipes to save command outputs into files, capture error logs, and chain multiple text-processing tools together.

Syntax

```
command > file | command >> file | command 2> err.txt | command1 | command2
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`>`	Overwrite output	Function: Creates or overwrites a file with the standard output. Example: **`cat file1.txt > file2.txt`**
`>>`	Append output	Function: Appends standard output to the end of an existing file. Example: **`cat >> hard-link.txt`**
`2>`	Redirect standard error	Function: Redirects error messages (stderr) to a file. Example: **`ls invalid 2> err.txt`**
`<`	Redirect standard input	Function: Redirects standard input from a file instead of the keyboard. Example: **`tr 'a-z' 'A-Z' < name.txt`**
`|`	Pipe operator	Function: Connects the standard output of one command directly to the standard input of another. Example: **`cat names.txt | wc -l`**

Practice terminal — test commands and actual output

Ubuntu terminal | redirection & pipes | lab practice

```
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ cat file1.txt > file2.txt
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ cat names.txt | wc -l 14
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ grep "Axmed Cali" names.txt | wc  1
2 11
```

3.4 sort, uniq, cut, paste, wc, and tr

Definition

`sort` arranges lines of text alphabetically by default. **`uniq`** removes or filters out adjacent duplicate lines. **`cut`** extracts specific fields, columns, or characters from text lines. **`paste`** combines corresponding lines from multiple files side-by-side. **`wc`** counts lines, words, and bytes, and **`tr`** translates, deletes, or squeezes individual characters from standard input.

Why students use this command

Use these text filters to clean CSV datasets, count records, deduplicate sorted lists, and standardize text formatting.

Syntax

```
sort [OPT] [FILE] | uniq [OPT] | cut -d [DELIM] -f [FIELDS] [FILE] | wc [OPT] [FILE] | tr [SET1] [SET2]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`sort -n / -r`	Numeric / Reverse sort	Function: Sorts lines numerically (`-n`) or in reverse order (`-r`). Example: `sort -rn num.txt`
`uniq -c / -d`	Count / Duplicates	Function: Counts consecutive occurrences (`-c`) or prints duplicates (`-d`). Example: `sort name.txt | uniq -c`
`cut -d -f`	Delimiter & field extraction	Function: Extracts specific fields using a delimiter. Example: `cut -d, -f1,2 test.txt`
`wc -l / -w / -c`	Line, word, byte count	Function: Counts lines (`-l`), words (`-w`), or bytes (`-c`). Example: `wc -l names.txt`
`tr 'a-z' 'A-Z'`	Character translation	Function: Translates lowercase letters to uppercase. Example: `tr 'a-z' 'A-Z' < name.txt`

Practice terminal — test commands and actual output

```
Ubuntu terminal | sort, uniq, cut, paste, wc, tr | lab practice
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ sort name.txt | uniq -c  2
ahmed 1 cali
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ cut -d, -f1,2 test.txt name,age
muscab,21 ali,24 fahad,20
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ paste name.txt num.txt ahmed 1 cali 1
ahmed 3
ahmed@ahmed-VMware-Virtual-
Platform:~/Desktop$ tr 'a-z' 'A-Z' < name.txt
AHMED CALI AHMED
```

3.5 tar, gzip, bzip2, xz, and zip

Definition

`tar` bundles and packages multiple files or directories into an archive. **`gzip / gunzip`**, **`bzip2 / bunzip2`**, and **`xz / unxz`** compress or decompress individual files, while **`zip / unzip`** creates and extracts cross-platform compressed ZIP archives.

Why students use this command

Use archiving and compression utilities to bundle project directories into backups and reduce storage or network transfer size.

Syntax

```
tar [OPTIONS] [ARCHIVE] [FILES] | gzip [FILE] | gunzip [FILE.gz] | zip [ARCHIVE.zip] [FILES]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`tar -czf`	Create gzip archive	Function: Creates a new compressed gzip archive. Example: **`tar -czf student.tar.gz student/`**
`tar -xzf`	Extract gzip archive	Function: Extracts files from a compressed gzip archive. Example: **`tar -xzf student.tar.gz`**
`tar -tf`	List archive contents	Function: Lists files inside a tar archive without extracting. Example: **`tar -tf file.tar`**
`gzip -k -9`	Keep & max compression	Function: Compresses file at maximum level (**`-9`**) while keeping original (**`-k`**). Example: **`gzip -k -9 cns.txt`**

Practice terminal — test commands and actual output

Ubuntu terminal | tar & gzip | lab practice

```
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ tar -cf file.tar names.txt
numbers.txt cns241.txt ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ tar -tf
file.tar names.txt numbers.txt cns241.txt
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ gzip -k -9
cns.txt
 ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ ls cns.txt* cns.txt cns.txt.gz
```

3.6 nano, vim, and ln

Definition

nano is a beginner-friendly terminal text editor with visible command shortcuts. **vim** is a powerful modal text editor designed for advanced command-line text editing. **ln** creates hard and symbolic links to files.

Why students use this command

Use **nano** or **vim** to edit scripts and configuration files in the terminal, and use **ln** to create shared file references or shortcuts.

Syntax

```
nano [FILE] | vim [FILE] | ln [OPTION] [TARGET] [LINK_NAME]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`ln [src] [dest]`	Hard link	Function: Creates a hard link sharing the same inode and data. Example: **`ln source-link.txt hard-link.txt`**
`ln -s [src] [dest]`	Symbolic link (soft link)	Function: Creates a shortcut pointing to the target filename. Example: **`ln -s test.txt link-shortcut.txt`**

Practice terminal — test commands and actual output

Ubuntu terminal | nano & ln (hard and soft links) | lab practice

```
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ ln source-link.txt hard-link.txt
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ ln -s test.txt link-shortcut.txt
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ ls -l link-shortcut.txt lrwxrwxrwx 1 ahmed ahmed 8 Nov 11 12:25 link-shortcut.txt -> test.txt
```

Chapter 4: Process, Service and System Management

4.1 ps, echo $$, top, htop, jobs, bg, fg, sleep, and gnome-calculator

Definition

`ps` displays currently running processes in the system (Process Status). **`echo $$`** outputs the Process ID (PID) of the current shell. **`top`** and **`htop`** display real-time system processes and resource usage. **`jobs`** lists background jobs in your current shell, **`bg`** resumes a suspended background process, **`fg`** brings a background process to the foreground, **`sleep`** delays execution for a specified time, and **`gnome-calculator`** opens the GNOME Calculator application.

Why students use this command

Use these commands to inspect active processes, identify parent-child PID relationships, and manage foreground and background tasks.

Syntax

```
ps [OPTIONS] | echo $$ | top | htop | jobs -l | bg %[JOB] | fg %[JOB] | sleep [SEC] &
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`ps -ef / ps aux`	All system processes	Function: Displays full details for all running system processes. Example: **`ps aux --sort=-%mem | head -n 5`**
`ps -o pid,ppid,comm`	Custom output columns	Function: Formats process listing to show PID, PPID, and command name. Example: **`ps -o pid,ppid,comm`**
`Ctrl+Z / &`	Suspend / Background run	Function: Suspends foreground job (**`Ctrl+Z`**) or starts in background (**`&`**). Example: **`sleep 100`** then **`bg %1`**
`jobs -l`	List jobs with PIDs	Function: Shows active and stopped shell jobs alongside their PIDs. Example: **`jobs -l`**

Ubuntu terminal | ps, jobs, bg, fg | lab practice

```
      ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ echo $$ 3990
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ ps -o pid,ppid,comm  PID PPID
COMMAND 3990 3967 bash 4058 3990 ps
 ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ jobs -l [1]+ 4365 Stopped sleep
100
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ bg %1 [1]+ sleep 100 &
```

4.2 kill, killall, pkill, pgrep, nice, and renice

Definition

`kill` terminates or sends signals to a process using its Process ID (PID). **`killall`** terminates all processes sharing a specified name, while **`pkill`** terminates processes based on name or attributes. **`pgrep`** searches for and displays the PID of a process by its name. **`nice`** starts a process with a specified priority level, and **`renice`** modifies the priority level of an already running process.

Why students use this command

Use these process control tools to locate unresponsive tasks, send termination signals, and adjust CPU scheduling priority.

Syntax

```
kill [SIGNAL] [PID] | pgrep -a [NAME] | killall [NAME] | nice -n [VAL] [CMD] |
renice -n [VAL] -p [PID]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`kill -l`	List signal names	Function: Lists all available termination and control signals (e.g., SIGTERM, SIGKILL). Example: **`kill -l`**
`pgrep -a`	Find PIDs with command	Function: Displays matching Process IDs along with full command arguments. Example: **`pgrep -a sleep`**
`nice -n [val]`	Set initial priority	Function: Launches a command with a modified niceness value (-20 to 19). Example: **`nice -n 10 sleep 300 &`**
`renice -n [val] -p`	Change running priority	Function: Adjusts priority of an active PID (root required to lower value). Example: **`sudo renice -n 5 -p 4582`**

Ubuntu terminal | kill, pgrep, nice, renice | lab practice

```
      ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ sleep 600 & [1] 4419
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ kill 4419 [1]+ Terminated sleep 600
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ nice -n 10 sleep 300 & [5] 4582
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ renice -n 15 -p 4582 4582 (process
ID) old priority 10, new priority 15
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ sudo renice -n 5 -p 4582 4582 (
process ID) old priority 15, new priority 5
```

4.3 hostnamectl, uptime, nproc, free, lscpu, and lsblk

Definition

`hostnamectl` displays or changes the system's hostname. **`uptime`** shows how long the system has been running, current time, logged-in users, and load averages. **`nproc`** prints the number of processing units (CPU cores) available. **`free`** displays the amount of free and used RAM memory in the system, **`lscpu`** provides detailed CPU architecture information, and **`lsblk`** lists block storage devices such as hard drives and partitions.

Why students use this command

Use these hardware and system diagnostics commands to audit RAM availability, CPU core counts, disk partitions, and system uptime.

Syntax

```
hostnamectl | uptime [OPT] | nproc | free [OPT] | lscpu | lsblk [OPT]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`uptime -p / -s`	Pretty / Boot timestamp	Function: Shows uptime in human format (`-p`) or exact boot time (`-s`). Example: `uptime -p`
`free -h / -m`	Memory usage units	Function: Displays RAM and swap in human-readable (`-h`) or megabytes (`-m`). Example: `free -h`
`lsblk -e 7`	Exclude loop devices	Function: Lists physical disks and partitions while hiding loop devices. Example: `lsblk -e 7 -d -o NAME,SIZE,MODEL`

Practice terminal — test commands and actual output

Ubuntu terminal | uptime, nproc, free, lsblk | lab practice

```
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ uptime -p up 45 minutes
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ nproc 2
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ free -h
total used free shared buff/cache available Mem: 3.3Gi 1.2Gi 402Mi 17Mi 1.9Gi 2.0Gi
Swap: 3.8Gi 0B 3.8Gi
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ lsblk -e 7 -d -o NAME,SIZE,MODEL
NAME SIZE MODEL fd0 4K sda 25G VMware Virtual S sr0 107.9M VMware Virtual SATA CDRW
Drive sr1 6G VMware Virtual SATA CDRW Drive
```

4.4 systemctl, journalctl, logger, last, dmesg, shutdown, reboot, poweroff, and sshd

Definition

`systemctl` is the primary command to manage services and the Systemd system and service manager. **`journalctl`** reads and queries system logs managed by the Systemd journal, while **`logger`** files messages into the system log. **`last`** displays a history of users who logged into the system, and **`dmesg`** prints the kernel ring buffer (driver and boot messages). **`shutdown`**, **`reboot`**, and **`poweroff`** safely restart or power off the system, and **`sshd`** is the Secure Shell daemon service that enables secure remote login connections.

Why students use this command

Use these administrative commands to start, stop, or inspect background daemons like **`cron`**, query system journal logs, and control system power states.

Syntax

```
systemctl [ACTION] [SERVICE] | journalctl [OPTIONS] | logger "[MSG]" | shutdown
| reboot | poweroff
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`systemctl status / is-active`	Inspect service state	Function: Checks detailed status or active state of a systemd unit. Example: **`systemctl status cron`**
`systemctl start / stop / restart`	Control service execution	Function: Starts, stops, or restarts a background daemon. Example: **`sudo systemctl restart cron`**
`journalctl -u [unit] -n [num]`	Filter service logs	Function: Displays the last N journal log entries for a specific unit. Example: **`journalctl -u cron -n 4`**
`journalctl -b -p err`	Boot error logs	Function: Shows error-priority messages from the current boot. Example: **`journalctl -b -p err`**

Ubuntu terminal | systemctl, journalctl, logger | lab practice

```
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ systemctl is-active cron
active
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ sudo systemctl stop cron ahmed@
ahmed-VMware-Virtual-Platform:~/Desktop$ systemctl is-active cron inactive ahmed@
ahmed-VMware-Virtual-Platform:~/Desktop$ sudo systemctl start cron
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ logger "week 8 test from student"
ahmed@ahmed-VMware-Virtual-Platform:~/Desktop$ journalctl -u cron -n 4 Nov 11
14:42:08 ahmed-VMware-Virtual-Platform systemd[1]: Stopping cron.service... Nov 11
14:42:08 ahmed-VMware-Virtual-Platform systemd[1]: cron.service: Deactivated
successfully. Nov 11 14:42:08 ahmed-VMware-Virtual-Platform systemd[1]: Started
cron.service - Regular background program processing daemon.
```

Conclusion

In conclusion, Linux commands are important tools for managing and controlling the Linux operating system. Through this presentation, we have learned different commands, their functions, and how to use them in practical situations. Understanding these commands helps us improve our Linux skills and manage the system more effectively.